

Orçamento Preliminar IFC

1. Visão geral

O **Orçamento Preliminar IFC** é um aplicativo web que automatiza a geração de um orçamento preliminar de obra a partir de um modelo **BIM** no formato **IFC** (Industry Foundation Classes). O usuário importa um arquivo `.ifc`, visualiza o modelo em 3D e obtém uma planilha orçamentária com serviços, quantidades, unidades e preços — exportável em Excel.

A premissa central é que o IFC, por ser um dado **estruturado**, já carrega as quantidades calculadas dos elementos construtivos (`BaseQuantities`). Isso elimina a calibração manual e os erros de medição típicos da extração a partir de plantas em PDF.

2. Objetivos

- Reduzir o tempo de levantamento de quantitativos (takeoff) de obras.
- Traduzir elementos BIM em **linguagem de orçamento** (serviço/unidade/quantidade).
- Precificar automaticamente com base em referência **SINAPI**, permitindo ajustes manuais pelo orçamentista.
- Entregar uma planilha pronta para uso em `.xlsx`.

3. Arquitetura e stack

Camada	Tecnologia	Responsabilidade
Front-end / UI	Next.js (App Router) + React + Tailwind	Upload, viewer 3D, tabela editável, exportação
Visualização 3D	web-ifc	Renderiza o modelo e destaca objetos
Parsing	Parser STEP próprio (<code>lib/ifc.ts</code>)	Lê elementos IFC e quantidades
Regra de negócio	<code>lib/budget.ts</code>	Traduz elementos em itens de orçamento
Precificação	<code>lib/sinapi.ts</code>	Tabela de referência de custos
Exportação	<code>exceljs (lib/ifcXlsx.ts)</code>	Gera a planilha <code>.xlsx</code>

3.1 Estrutura de pastas

<code>app/page.tsx</code>	<code># UI: upload IFC, viewer 3D, tabela e download</code>
<code>app/layout.tsx</code>	<code># metadados e fontes</code>
<code>app/api/ifc/route.ts</code>	<code># endpoint: parse do IFC -> orçamento (JSON)</code>
<code>app/api/ifc/export/route.ts</code>	<code># endpoint: gera a planilha .xlsx</code>
<code>lib/ifc.ts</code>	<code># parser STEP + extração de quantidades + unidades</code>
<code>lib/budget.ts</code>	<code># tradução para itens de orçamento (PT-BR)</code>
<code>lib/sinapi.ts</code>	<code># tabela de referência de custos SINAPI</code>
<code>lib/ifcXlsx.ts</code>	<code># montagem da planilha (exceljs)</code>

4. Metodologia de desenvolvimento com agentes de IA

O projeto foi desenvolvido com o apoio de **agentes de IA especializados**, cada um responsável por uma frente do trabalho, simulando um time de software multidisciplinar:

Agente	Papel	Responsabilidades
Agente de Backend	Desenvolvimento server-side	Parser do IFC, endpoints da API (<code>/api/ifc</code> , <code>/api/ifc/export</code>), normalização de unidades e geração da planilha.
Agente de Frontend	Interface e experiência	UI em Next.js/Tailwind, visualizador 3D, tabela editável, fonte de custos e identidade visual.
Agente de QA	Qualidade e validação	Verificação de tipos, testes com o modelo de referência, conferência de quantidades/unidades e revisão de regressões.
Agente Orçamentista	Domínio de orçamento	Tradução dos elementos IFC em itens de serviço, critérios de engenharia (faces de parede, cobertura, estimativas) e mapeamento de preços SINAPI com unidades compatíveis.

Essa divisão por papéis permitiu separar claramente as preocupações técnicas (backend, frontend, QA) da regra de negócio de orçamentação, conduzida pelo **agente orçamentista**, que define como o modelo BIM se traduz em custos.

5. Fluxo de dados

- Upload** — o usuário seleciona um `.ifc` na interface.
- Parse** (`POST /api/ifc`) — o arquivo é lido pelo parser STEP, que extrai os elementos (paredes, lajes, pilares, vigas, fundações, portas, janelas, coberturas) e suas quantidades.
- Orçamentação** (`lib/budget.ts`) — os elementos viram itens de serviço agrupados por etapa, com unidade e quantidade.
- Precificação** (`lib/sinapi.ts`) — cada item recebe um preço unitário de referência SINAPI (ou inserido pelo usuário).
- Visualização** — a tabela é exibida na web; o modelo aparece em 3D.
- Exportação** (`POST /api/ifc/export`) — gera o `.xlsx` com os preços efetivos.

6. Parser IFC (`lib/ifc.ts`)

O IFC segue o padrão **STEP / ISO-10303-21**. O parser implementado é leve e não usa WASM. Principais responsabilidades:

- Leitura das entidades** (`#id = IFCTIPO(args)`) e dos conjuntos de quantidades (`IfcElementQuantity / BaseQuantities`).
- Normalização de unidades**: comprimentos `mm`→`m`, áreas `mm²`→`m²`, volumes `mm³`→`m³`, com heurística por magnitude do valor.
- Deduplicação** de quantidades repetidas pelo exportador (mantém o valor líquido / mínimo coerente).
- Fallback geométrico**: quando um elemento não tem `BaseQuantities` , tenta derivar a quantidade a partir da geometria (ex.: `IfcExtrudedAreaSolid + IfcRectangleProfileDef`).

7. Regras de orçamento (`lib/budget.ts`)

Os elementos IFC são convertidos em itens agrupados por etapa. Resumo:

Etapa	Itens (exemplos)	Unidade
1. Serviços preliminares	Limpeza do terreno, locação da obra	m², m
2. Fundações	Concreto, fôrmas, armadura	m³, m², kg

Etapa	Itens (exemplos)	Unidade
3. Superestrutura	Concreto em pilares/vigas/lajes, fôrmas, armadura	m³, m², kg
4. Alvenaria/Vedações	Paredes externas e internas	m²
5. Cobertura	Telhamento (inclui estrutura)	m²
6. Esquadrias	Portas, janelas e seus vãos	un, m²
7. Revestimentos	Chapisco, emboço, reboco	m²
8. Pisos	Contrapiso, revestimento de piso	m²
9. Pintura	Paredes e teto/forro	m²

Critérios de engenharia adotados:

- **Paredes** classificadas em externa/interna por palavra-chave do nome; área de revestimento considera 1 face para externas e 2 faces para internas.
- **Cobertura** usa a **face inferior** das lajes de telhado (forro) — não a superfície total da laje (que somaria faces e bordas).
- **Pintura de teto** só aparece quando existe laje de teto no modelo.
- Itens marcados como **(estimado)** derivam de premissas paramétricas (ex.: ~10 m² de fôrma por m³ de concreto, ~100 kg/m³ de armadura).

8. Precificação SINAPI ([lib/sinapi.ts](#))

O app oferece uma **Fonte de Custos** selecionável no topo:

- **SINAPI (padrão)** — preços de referência (estado de **SP**, regime **não desonerado**) alimentam a coluna de preço unitário. Cada item do orçamento é mapeado para a composição SINAPI mais próxima, garantindo **unidade compatível**. A interface exibe o código/descrição da composição usada.
- **Inserido pelo Usuário** — todos os preços são digitados manualmente.

No modo SINAPI, o usuário ainda pode **editar** o preço de um item específico (que então deixa de mostrar o código SINAPI); o botão **↶ SINAPI** devolve aquele item ao valor de referência.

Os valores SINAPI embutidos são **referências aproximadas**. O SINAPI oficial é publicado mensalmente pela Caixa por UF; recomenda-se revisar os números de [lib/sinapi.ts](#) conforme o mês/estado do orçamento.
<https://www.caixa.gov.br/site/Paginas/downloads.aspx>

9. Exportação Excel ([lib/ifcXlsx.ts](#))

A planilha `.xlsx` é montada com **exceljs** e contém:

- **Orçamento** — código, etapa, serviço, unidade, quantidade, preço unitário, total e critério/observação, com linha de total geral.
- **Detalhe** — lista por elemento (tipo, nome, GUID, área, comprimento, volume).

10. Visualizador 3D

O modelo IFC é renderizado com **web-ifc**. Cada item do orçamento pode **destacar** no modelo os objetos paramétricos que originaram sua quantidade, facilitando a conferência visual do takeoff.

11. Como executar

```
npm install
npm run dev      # http://localhost:3000
```

```
npm run build
npm run typecheck
```

Requer Node.js 18+ (testado com Node 24).

Uso

1. Abra `http://localhost:3000`.
2. Clique em **Abrir IFC** e selecione um arquivo `.ifc`.
3. Confira o modelo 3D e a planilha; ajuste a **Fonte de Custos** e os preços.
4. Clique em **Exportar Excel**.

12. Validação

Testado com o modelo **BasicHouse.ifc** (IFC2X3 exportado do Revit). As quantidades e os preços foram conferidos item a item, validando a compatibilidade de unidades entre o app e a referência SINAPI.

13. Limitações e próximos passos

- Quantificação **por elemento**, não por ambiente (o IFC de teste não traz `IfcSpace`). Suporte a área por cômodo pode ser adicionado quando houver espaços no modelo.
- Classificação externa/interna de paredes por **palavra-chave** do nome.
- Referência SINAPI **estática** e regional (SP) — evoluir para múltiplas UFs e seleção do mês de referência.
- Possível adicionar BDI, encargos e composição detalhada de preços.